

BARE METAL / SHARP X68000

PONG

Human68k非依存のC実装
1P / 2P ・ キーボード / パッド
対応

elf2x68k + IOCS + mode 12

Makefile -> dist/pong.xdf



GameContextがアプリケーションと各モードを束ねる

ApplicationState

- old_mode
起動前の画面モード
- mode : GameModelId
現在の画面状態
- application_loop()
GameModeの遷移を一元管理

GameContext

application : ApplicationState

title : TitleState

pong : PongGameState

winner : WinnerState

共有コンテキストに状態を集約

makeだけでベアメタルXDFまで生成

01 /
COMPILE

m68k-xelf-gcc

-m68000 -O2
-specs=x68knodos.specs
リンク先 0x6800

02 /
BOOT

human.sysへ配置

リンク時にhuman.sysを直接生成。
Human68k本体やシステムファイル
は不要。

03 / XDF

dist/pong.xdf

human.sysだけを格納した
新しいXDFイメージを生成。

512 x 480 / 60 Hz表示

MODE

SYNC

TIMING

CRTMOD(12)

512 x 512 / 65536色の
標準31 kHzモードを選択。

wait_vdisp()

MFP GPIPのV-DISPエッジを待ち
、
1更新を1フレームに固定。

R05/R06/R07 -> R04

表示位置を先に設定し、
最後に総走査線525へ変更。

_ONTIMEで待機タイムアウトを監視し、停止を回避

GameModeが画面のライフサイクルを統一

```
typedef struct {
    void (*initialize)(GameContext *);
    GameModeId (*update)(GameContext *);
    void (*finalize)(GameContext *);
} GameMode;

next =
game_modes[current].update(context);
if (next != current) {
    game_modes[current].finalize(context);
    game_modes[next].initialize(context);
}
```

遷移をapplication_loop()へ集約

- 01 initialize()で画面へ入る
- 02 update()は1フレームだけ処理
- 03 戻り値で次のGameModeIdを指定
- 04 finalize()で画面を離れる
- 05 title / pong / winnerを同じ形で管理

新規GameModeは状態・3関数・モード表1行で追加。

キーボードと2台のパッドに対応

PLAYER 1
W / S
PAD 1 UP / DOWN

PLAYER 2
CURSOR UP / DOWN
PAD 2 UP / DOWN

TITLE

上下で選択、Return / Space / Aで決定

MATCH

Q / ESCでタイトルへ戻る

STATUS KEYBOARD + 2 PADS

公開リポジトリを用途別に整理

01

src/

pong.c と Makefile。
Cソースとビルド定義を集約。

02

dist/

makeで生成する起動ディスク
pong.xdfを配置。

03

docs/ + README

技術資料はpong-ja.pdf。
READMEは概要とビルド手順。